Laya in HyperSpace & La Selezione dei Nodi

Decision Engine System 1 non-autoregressivo e Routing a 2 Stadi (Semantico vs Metrico)

MODULO: LAYA SYSTEM 1	LATENZA: SUB-50 MS	ROUTING A 2 STADI	HARDWARE: MPS / MESH

1. Cosa fa Laya adesso in HyperSpace (Recap)

Laya (NandhaKishorM/laya) agisce come il livello **System 1 (intuitivo, categorico, sub-50ms)** di HyperSpace, perfettamente complementare al **System 2 (deliberativo, autoregressivo, CoT lento)**:

- **1. Fast-Gate in Mission Control (control-plane/cp/mission.py):**
  Quando un worker completa un task, il verifier deve emettere un verdetto rapido: pass, retry, replan o fail. Invece di attendere 5–10 secondi e spendere token invocando un LLM grande, **Laya valuta la conformità del task in ~35 ms** in singola forward pass. Se la confidenza è ≥ 0.85, il verdetto è registrato subito (system: 'laya-1'). Se Laya è incerto o offline, scatta in automatico il fallback su System 2 (_mission_llm).
- **2. Client tipizzato fail-safe (shared/laya_client.py):**
  Mette a disposizione di tutto il repository chiamate pure (decide_choice, decide_noul, decide_score, predict_batch) senza appesantire il .venv di base con PyTorch o transformers. Gestisce soglie di astensione (abstention) e fail-closed sicuro.
- **3. Infrastruttura e Diagnostica:**
  È orchestrabile via Docker con profilo opzionale (docker compose --profile decision up -d laya) o tramite il runner locale Apple Silicon Metal (scripts/laya_service.py --device mps). Monitorabile in qualsiasi momento con lo script di diagnostica scripts/diagnostics/laya_health.py.

2. Come vengono scelti i nodi? (Il Routing a 2 Stadi)

In HyperSpace la scelta del nodo è un processo a **due stadi separati e complementari**:
Stadio 1: Laya decide *cosa serve alla richiesta* (Routing Semantico & di Capacità).
Stadio 2: HyperSpace Control Plane decide *chi ha le risorse fisiche per farlo* (Routing Hardware & Metriche).

Richiesta Utente (Prompt / Task)

↓

STADIO 1: LAYA (Routing Semantico & Triage Istantaneo ~33ms)
- Rilevamento Lingua & Script (<0.5ms): Inglese vs Multilingua (100+ lingue)
- Identificazione Classe Requisiti: code_sandbox, multimedia_vision, fast_chat, reasoning_heavy
- Flag Reasoning Dinamico: think = true / false (spegne il CoT se bastano i tool)

↓ [Requisiti Estratti: classe modello, memoria richiesta, tool abilitati]

STADIO 2: HYPERSPACE CONTROL PLANE (Routing Hardware & Metriche — control-plane/routing.py)
- Normalizzazione Min-Max dei candidati idonei della flotta mesh
- Punteggio di Qualità Dinamico: Latenza EWMA + Throughput TPS effettivo + Pressione cache GPU
- Punteggio Strutturale: VRAM disponibile (peso 55%) + Saturazione/Carico (25%) + Tier (10%)
- Gate di Salute (health_s) & Penalità Anti-Hotspot Esponenziale

↓

Mac Locale (Apple Silicon)	**Worker Remoto (GPU RTX / Linux)**
Control Plane, Laya su Metal (MPS), Tool leggeri, Triage	ComfyUI, Generazione Immagini, Modelli grandi 32B/70B

Stadio 1 — Come Laya instrada semanticamente

Laya opera su due livelli decisionali interni prima che qualsiasi byte tocchi la rete:

- **Router dei Checkpoint Interno (<0.5 ms):** Laya analizza lo script e la lingua all'istante:
  – Testo inglese → checkpoint laya (ModernBERT-large, 421M);
  – Testo italiano o altre 100+ lingue → checkpoint laya-multilingual (mmBERT, 322M);
  – Workflow strutturati specifici → checkpoint laya-typed-decisions.
- **Triage di Classe e Requisiti:** Con una forward pass categorica choice, Laya classifica il fabbisogno:
  – code_sandbox: richiede un nodo con runner Docker o sandbox isolata;
  – multimedia_vision: instrada verso il worker dedicato alle immagini (ComfyUI con alta VRAM);
  – fast_chat: gestibile dal modello leggero locale senza allocare risorse pesanti;
  – reasoning_heavy: richiede un nodo con modello di deduzione logica (Qwen/DeepSeek CoT attivo).

Stadio 2 — Come HyperSpace sceglie la macchina fisica (control-plane/routing.py)

Una volta che i requisiti sono noti, entra in gioco il router metrico di HyperSpace (compute_score), che calcola un punteggio matematico normalizzato per ciascun nodo candidato:

$$\text{Score} = \text{Health_Gate} \times (q \cdot \text{Quality} + (1 - q) \cdot \text{Structural}) - \text{Recent_Penalty}$$

- **Gate di Salute (Health_Gate):** Se il nodo è online e invia metriche fresche: moltiplicatore 1.0. Se è degradato o i campioni sono vecchi: depenalizzato a 0.4. Se offline o non risponde: 0.0 (eliminato).
- **Punteggio di Qualità (Quality):** Latenza EWMA (lat_s, peso 45%), Throughput (tps_s, peso 35%) e Pressione cache GPU (gpu_s, peso 20%). Il fattore *q* indica la freschezza delle metriche (1.0 se fresche, 0.0 se assenti).
- **Punteggio Strutturale (Structural):** Entra in gioco quando i campioni latenza mancano. **VRAM libera (vram_s, peso 55%)** dà priorità alle macchine con memoria video capiente (fondamentale per il nodo remoto ComfyUI); Carico/Saturazione (load_s, peso 25%); Tier di rete gerarchico root > hub > leaf (peso 10%).
- **Penalità Anti-Hotspot (Recent_Penalty):** Decremento esponenziale applicato al nodo selezionato più di recente per prevenire il sovraccarico di una singola macchina e distribuire il traffico equamente.

3. In Sintesi: Matrice di Responsabilità

Dimensione	Laya (System 1)	HyperSpace Router (routing.py)
Scopo Primario	**Routing Semantico** e Decisioni Tipizzate	**Routing Hardware** e Bilanciamento di Carico
Tempo di Risposta	**30–35 ms** (GPU) / ~120 ms (Mac MPS)	**< 5 ms** (Calcolo matematico puro su telemetria)
Input Analizzato	Testo, prompt, evidenza task, lingua, intent	Heartbeat, VRAM (GB), saturazione, latenza EWMA, TPS
Output Prodotto	Scelta categorica (choice), booleano (noul), score	ID del nodo fisico ottimale nella mesh
Comportamento Fallimento	Astensione calibrata → escalation a System 2	Failover automatico al nodo secondo classificato

Conclusione Architetturale: Laya e HyperSpace routing.py lavorano in tandem armonico. Laya è il *cervello di triage rapido* che capisce in 30 ms il bisogno della richiesta o l'esito del lavoro, mentre routing.py è il *direttore infrastrutturale* che invia il calcolo pesante al nodo più idoneo della mesh (Mac locale per l'interazione rapida, worker GPU remoto per ComfyUI e rendering video).